

Trabalho Prático

Igor Henrique Martins de Almeida - 2023028536

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte - MG - Brasil

igorhma@ufmg.br

1. Introdução

O trabalho prático consistiu no desenvolvimento de três programas distintos na linguagem C. Cada programa deveria ser capaz de determinar e imprimir coordenadas cartesianas (x, y) de um ponto em uma espiral bidimensional, dado um número inteiro não negativo n que representa a ordem desse ponto na sequência da espiral. A seguinte documentação foi separada em 6 seções:

- Introdução
- Método
- Desenvolvimento
- Análise de Complexidade
- Conclusão
- Referências Bibliográficas

2. Método

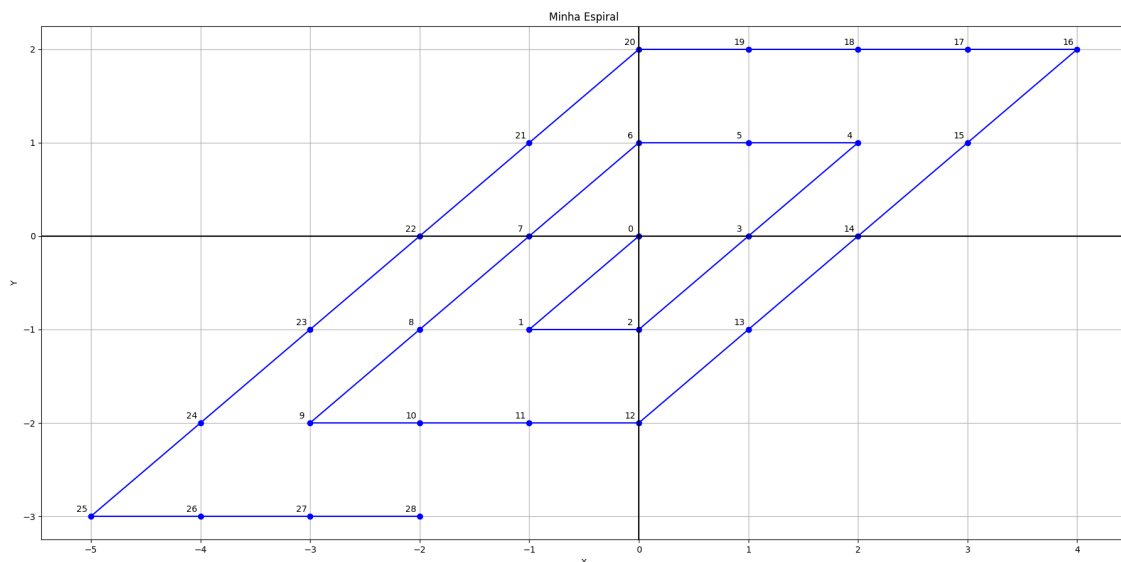
A abordagem metodológica comum para resolver o problema em todas as três espirais consistiu em:

- **Análise de Padrões:** Observação das figuras e tabelas de coordenadas fornecidas no enunciado do trabalho prático (para as espirais quadrada e triangular) e definição de um novo padrão (para a "Minha Espiral").
- **Identificação de Pontos de Referência:** Em todos os casos, os números que são quadrados perfeitos foram identificados como pontos-chave ou "cantos" nas espirais. A estratégia principal envolveu calcular as coordenadas para o quadrado perfeito mais próximo do número de entrada `coord`.
- **Cálculo de Coordenadas para Quadrados Perfeitos:** Foram deduzidas fórmulas ou regras específicas para determinar as coordenadas (x,y) desses

quadrados perfeitos, geralmente levando em consideração a paridade do quadrado perfeito.

- **Determinação da Direção de "Caminhada":** Uma vez localizadas as coordenadas do quadrado perfeito mais próximo, foi necessário determinar a direção em que a espiral "caminha" para alcançar o ponto `coord`. Essa direção foi codificada em uma variável `direcao`.
- **Cálculo do Deslocamento:** A diferença absoluta entre `coord` e o `quadradoPerfeito` mais próximo (`diferenca`) foi calculada para determinar quantos "passos" seriam necessários para alcançar o ponto final.
- **Ajuste Final das Coordenadas:** Utilizando uma estrutura `switch` baseada na `direcao`, as coordenadas (x,y) do quadrado perfeito foram ajustadas pela `diferenca` para encontrar as coordenadas do ponto `coord`.

Para a "Minha Espiral", foi necessário conceber um novo padrão que não fosse uma simples variação das duas espirais fornecidas. A espiral projetada também utiliza a lógica dos quadrados perfeitos como pontos de ancoragem, mas com regras de posicionamento e movimentação distintas. A imagem abaixo ilustra como foi pensada a nova espiral:



3. Desenvolvimento

Todos os programas foram desenvolvidos na linguagem C e utilizam bibliotecas padrão como `stdio.h` para entrada e saída de dados (funções `scanf` e `printf`), `stdlib.h` para funções gerais como `abs` (valor absoluto), e `math.h` para operações matemáticas como `sqrt` (raiz quadrada), `round` (arredondamento), `floor` (piso) e `ceil` (teto).

As operações realizadas em cada um dos códigos incluem:

- Ler o número do ponto desejado (`coord`) fornecido pelo usuário.
- Calcular a raiz quadrada de `coord` (`raiz`).
- Determinar o quadrado perfeito (`quadradoPerfeito`) mais próximo de `coord` utilizando `round(raiz) * round(raiz)`.
- Calcular as coordenadas (`x`, `y`) para este `quadradoPerfeito` com base em padrões identificados para cada tipo de espiral e na paridade do `quadradoPerfeito`.
- Definir uma `direcao` de movimento na espiral. Essa direção é determinada por regras específicas para cada espiral, geralmente considerando a paridade do `quadradoPerfeito` e a posição relativa de `coord` em relação a ele (se `coord` é maior ou menor, ou em que "segmento" do braço da espiral ele se encontra).
- Se `coord` não for um quadrado perfeito, calcular a `diferenca = abs(quadradoPerfeito - coord)`.
- Ajustar as coordenadas `x` e `y` com base na `direcao` e na `diferenca`, utilizando uma estrutura `switch` para aplicar a movimentação correta na espiral.
- Imprimir as coordenadas finais (`x`, `y`).

Em resumo, os três algoritmos decompõem o problema de encontrar a coordenada de um ponto na espiral localizando primeiro um ponto de referência (o quadrado perfeito mais próximo) e, em seguida, "caminhando" a partir desse ponto de referência até o ponto desejado, seguindo as regras de construção de cada espiral.

4. Análise de complexidade

Para todos os casos, assume-se que a função `sqrt()` (cálculo da raiz quadrada) possui um custo computacional constante, $\Theta(1)$. A análise de complexidade de tempo dos algoritmos desenvolvidos para as três espirais (`espquadrada.c`, `esptriangular.c` e `minhaesp.c`) é **$\Theta(1)$ (tempo constante)**.

A justificativa para essa complexidade reside no fato de que todos os algoritmos executam um número fixo de operações, independentemente do valor de entrada `coord` (o número do ponto na espiral). As operações detalhadas foram explicitadas na seção anterior.

É importante notar que nenhum dos algoritmos utiliza laços de repetição (como `for` ou `while`) ou chamadas recursivas cuja execução depende do valor de `coord`. Dessa forma, o tempo de execução não aumenta proporcionalmente ao valor da entrada, caracterizando uma complexidade de tempo constante.

4.1. `espquadrada.c`

O algoritmo para a espiral quadrada segue os passos mencionados acima:

- Lê o inteiro `coord`.
- Calcular `raiz = sqrt(coord)`.
- Determina `quadradoPerfeito = round(raiz) * round(raiz)`.
- Com base na paridade de `quadradoPerfeito` e se `quadradoPerfeito` é menor ou maior que `coord`, define as coordenadas iniciais de `x`, `y` e uma `direcao` para ajuste. Essas são operações diretas.
- Se `quadradoPerfeito != coord`, calcula a `diferenca = abs(quadradoPerfeito - coord)` e ajusta para `x` ou `y` com base na `direcao` usando um `switch`. Cada case do `switch` realiza uma única operação de adição ou subtração.
- Imprime `(x, y)`.

Todas as etapas são executadas em tempo constante. Portanto, a complexidade do `espquadrada.c` é $\Theta(1)$.

4.2. `esptriangular.c`

O algoritmo para a espiral triangular possui uma estrutura semelhante:

- Lê o inteiro `coord`.
- Calcula `raiz = (coord)`.
- Determina `quadradoPerfeito = round(raiz) * round(raiz)`.
- Com base na paridade de `quadradoPerfeito` e na condição `raiz - floor(sqrt(coord)) < 0.5`, define as coordenadas iniciais `x`, `y` e uma `direcao`.
- Se `quadradoPerfeito != coord`, calcula `diferenca = abs(quadradoPerfeito - coord)` e ajusta `x` e/ou `y` com base na `direcao` usando um `switch`. Cada case realiza um número fixo de operações aritméticas.
- Imprime `(x, y)`

Novamente, todas as etapas são de tempo constante. Assim, a complexidade do `esptriangular.c` é $\Theta(1)$.

4.3. `minhaesp.c`

O algoritmo para a espiral personalizada segue uma lógica muito similar à da espiral triangular, diferenciando-se principalmente nas regras de movimentação e cálculo das coordenadas.

- Lê o inteiro `coord`.
- Calcular `raiz = sqrt(coord)`.
- Determina `quadradoPerfeito = round(raiz) * round(raiz)`.
- Com base na paridade de `quadradoPerfeito` e na condição `raiz - floor(sqrt(coord)) < 0.5`, define as coordenadas iniciais `x`, `y` e uma `direcao`.

- Se `quadradoPerfeito != coord`, calcula `diferenca = abs(quadradoPerfeito - coord)` e ajusta `x` e/ou `y` com base na `direcao` usando um switch.
- Imprime `(x, y)`.

A estrutura do algoritmo `minhaesp.c` é idêntica aos anteriores em termos de fluxo de controle e tipos de operações realizadas. Cada etapa é executada em tempo constante. A justificativa para a complexidade $\Theta(1)$ é a mesma: o número de operações não depende do valor de `coord`. Mesmo que as regras específicas para determinar `x`, `y` e a movimentação sejam diferentes, elas ainda consistem em um número fixo de cálculos aritméticos e comparações.

Portanto, a complexidade do `minhaesp.c` também é $\Theta(1)$.

5. Conclusão

O presente trabalho prático culminou no desenvolvimento de três programas em linguagem C, cada um capaz de calcular e fornecer as coordenadas cartesianas (x,y) de um ponto em uma espiral bidimensional específica: a espiral quadrada, a espiral triangular e uma espiral original proposta. Conforme detalhado, a abordagem metodológica comum para todas as espirais envolveu a identificação de quadrados perfeitos como pontos de referência, o cálculo de suas coordenadas e, subsequentemente, o ajuste dessas coordenadas com base na diferença para o ponto desejado.

A realização deste trabalho prático proporcionou uma valiosa oportunidade para aplicar conceitos de matemática discreta na resolução de problemas algorítmicos. O processo de analisar os padrões visuais das espirais e traduzi-los em lógica programável foi particularmente instrutivo, reforçando a importância da observação atenta e da decomposição de problemas complexos em etapas menores e gerenciáveis. A concepção da "Minha Espiral" exigiu um exercício adicional de criatividade e especificação de regras, diferenciando-se das espirais fornecidas.

Um dos principais desafios encontrados foi a dedução precisa das fórmulas para as coordenadas dos quadrados perfeitos e, especialmente, a lógica para determinar a

direcao correta de "caminhada" na espiral e os ajustes correspondentes nas coordenadas. Em alguns momentos, visualizar o comportamento da espiral para números maiores e garantir que a lógica se mantivesse consistente exigiu depuração e testes cuidadosos. A restrição de criar uma nova espiral que não fosse uma simples variação das existentes também apresentou um desafio interessante, incentivando a exploração de diferentes formas de progressão no plano cartesiano. Em resumo, o trabalho permitiu solidificar a compreensão sobre como padrões matemáticos podem ser modelados computacionalmente e reforçou habilidades em desenvolvimento de algoritmos e programação em C. Os objetivos propostos foram alcançados, com os programas funcionando conforme o esperado e respeitando os requisitos de complexidade.

6. Referências Bibliográficas

COMMON mathematical functions. **cppreference.com**, 2024. Disponível em: <https://en.cppreference.com/w/c/numeric/math.html>. Acesso em: 18 maio 2025.